

实验题目：计数器和序列检测器的设计

班级：无 32

学号：2023010504

姓名：张乐

日期：2025 年 3 月 28 日

一、实验目的

1. 掌握简单时序逻辑电路的设计方法
2. 了解任意进制计数器的设计方法
3. 掌握有限状态机的实现原理和方法
4. 掌握序列检测的方法

二、设计方案

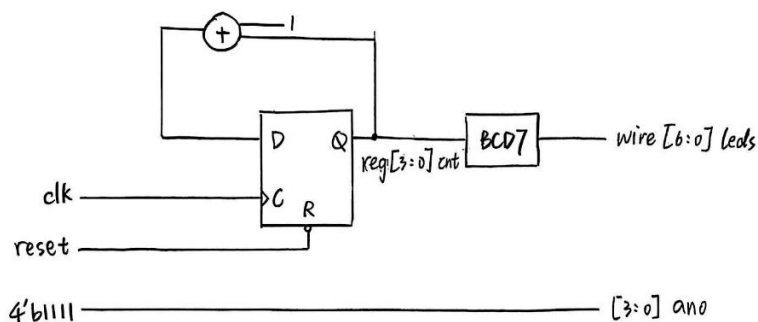
1. 具有异步复位控制的 4bits 十进制同步加法计数器

(1) 原理说明

计数器是一种常用的时序电路，它按照规定的方式改变内部各触发器的状态，以记录输入的时钟脉冲的个数。在同步加法计数器中，各个触发器使用同一个计数控制时钟，每一位在时钟上升沿到来时是否翻转取决于比其低的位是否都是“1”。其中，触发器的翻转是在时钟上升沿同步进行的，其翻转稳定时间仅仅取决于单级触发器的翻转时间，而与计数器的位数无关。而异步复位是指当复位信号为低电平时，计数器输出信号的所有位均置零。

将四位八段数码管作为计数器的输出显示，拨码开关做异步复位信号，1 个按键开关做时钟信号。

(2) 框图



2. 用有限状态机设计序列检测器

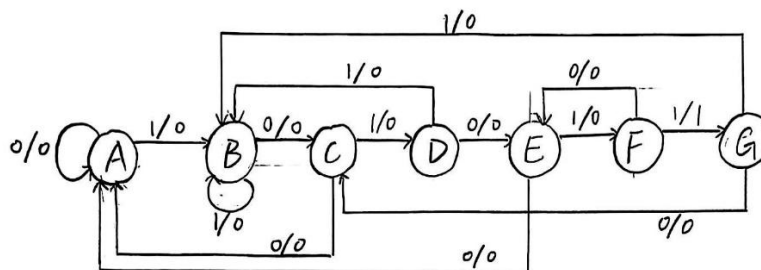
(1) 原理说明

有限状态机（Finite State Machine, FSM）常用于时序数字逻辑的设计。在复杂数字系统设计中，有限状态机主要通过硬件描述语言实现，硬件描述语言能够清晰的描述状态转移过程和输入输出变量关系，使得时序逻辑设计大大简化，进而极大降低系统设计复杂度，提高系统模块化程度。有限状态机从本质上讲是由寄存器和组合逻辑构成的时序电路，各个状态之间的转移总是在时钟的触发下进行的。为检测特定序列“101011”，可将检测过程分为 7

个状态，在时钟触发和输入的共同作用下实现序列检测。

将第 6~第 8 个 LED 作为状态机的编码显示，第 1 个 LED 做检测状态输出，拨码开关作为串行数据输入，2 个按键开关分别做异步复位信号和按键时钟输入。

(2) 框图



状态	状态编码	状态说明
A	000	完全未匹配到子序列
B	001	匹配到子序列“1”
C	010	匹配到子序列“10”
D	011	匹配到子序列“101”
E	100	匹配到子序列“1010”
F	101	匹配到子序列“10101”
G	110	匹配到序列“101011”，匹配成功

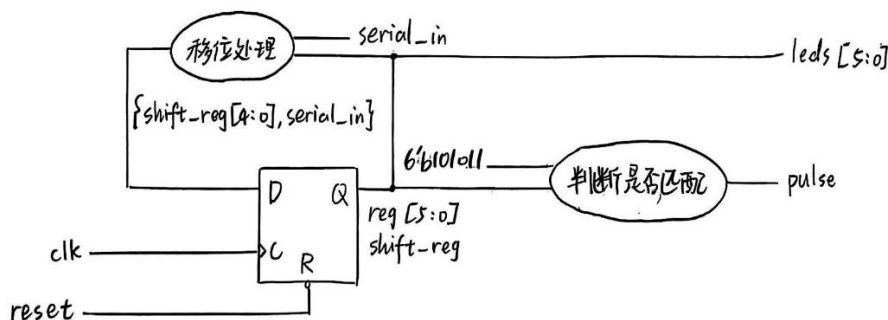
3. 用移位寄存器和组合逻辑实现序列检测器

(1) 原理说明

将移位寄存器初始化为 6b'000000，每输入 1 个信号，就将原序列左移一位，将输入信号补到原序列末尾，即用移位寄存器保留最后输入的 6 个信号。当移位寄存器保存的序列与目标序列“101011”完全一致时，匹配成功，输出宽度为 1 个时钟周期的高电平脉冲。

将第 2~第 7 个 LED 作为移位寄存器的状态显示，第 1 个 LED 做检测状态输出，拨码开关作为串行数据输入，2 个按键开关分别做异步复位信号和按键时钟输入。

(2) 框图



三、 关键代码及文件清单

1. 具有异步复位控制的 4bits 十进制同步加法计数器

(1) 关键代码

```
23 module counter(  
24     input clk,  
25     input reset,  
26     output reg [3:0] cnt,  
27     output wire [6:0] leds,  
28     output [3:0] ano  
29 );  
30  
31 assign ano=4'b1111;  
32  
33 always @(negedge reset or posedge clk)  
34 begin  
35     if(~reset) begin  
36         cnt<=4'b0000;  
37     end  
38     else begin  
39         if(cnt==4'b1001) cnt<=4'b0000;  
40         else cnt<=cnt+1;  
41     end  
42 end  
43  
44 BCD7 bcd27seg (cnt, leds);  
45  
46 endmodule
```

(2) 文件清单

文件名	功能
counter.v	主要文件
BCD7.v	四位八段数码管的显示文件
counter.xdc	约束文件

2. 用有限状态机设计序列检测器

(1) 关键代码

状态机时序逻辑部分

```
33 always @(posedge clk or posedge reset) begin
34     if (reset) begin
35         current_state<=3'b000;
36         pulse<=1'b0;
37     end else begin
38         current_state<=next_state;
39         pulse<=(next_state==3'b110)?1'b1:1'b0;
40     end
41 end
```

状态机组合逻辑部分

```
43 always @(*) begin
44     case (current_state)
45     3'b000: begin
46         if (serial_in==1'b1) next_state = 3'b001
47         else next_state=3'b000;
48     end
49     3'b001: begin
50         if (serial_in==1'b0) next_state=3'b010;
51         else next_state=3'b001;
52     end
53     3'b010: begin
54         if (serial_in==1'b1) next_state=3'b011;
55         else next_state=3'b000;
56     end
57     3'b011: begin
58         if (serial_in==1'b0) next_state=3'b100;
59         else next_state=3'b001;
60     end
61     3'b100: begin
62         if (serial_in==1'b1) next_state=3'b101;
63         else next_state=3'b000;
64     end
65     3'b101: begin
66         if (serial_in==1'b1) next_state=3'b110;
67         else next_state=3'b100;
68     end
```

```

69  3'b110: begin
70      if (serial_in==1'b0) next_state=3'b010;
71      else next_state=3'b001;
72  end
73      default: next_state=3'b000;
74  endcase
75  end

```

(2) 文件清单

文件名	功能
detector.v	主要文件
LED.v	LED 的显示文件
detector.xdc	约束文件

3. 用移位寄存器和组合逻辑实现序列检测器

(1) 关键代码

序列移位

```

31  always @(posedge clk or posedge reset) begin
32      if (reset) begin
33          shift_reg<=0;
34          pulse<=1'b0;
35      end else begin
36          shift_reg={shift_reg[4:0], serial_in};
37          pulse<=(shift_reg==6'b101011)?1'b1:1'b0;
38      end
39  end
40
41  endmodule

```

(2) 文件清单

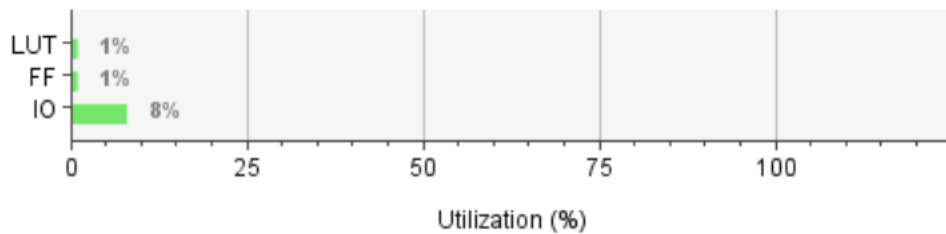
文件名	功能
detector2.v	主要文件
detector2.xdc	约束文件

四、 综合情况

1. 具有异步复位控制的 4bits 十进制同步加法计数器

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Bonded IOB (250)	BUFGCTRL (32)
counter	7	4	4	7	3	21	1

Resource	Utilization	Available	Utilization %
LUT	7	20800	0.03
FF	4	41600	0.01
IO	21	250	8.40

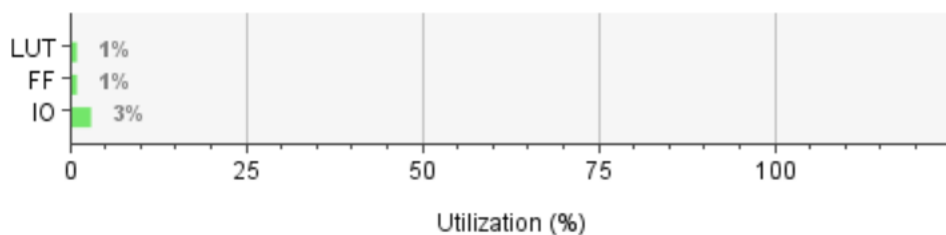


Setup	Hold	Pulse Width
Worst Negative Slack (WNS): inf	Worst Hold Slack (WHS): inf	Worst Pulse Width Slack (WPWS): NA
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): NA
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: NA
Total Number of Endpoints: 23	Total Number of Endpoints: 23	Total Number of Endpoints: NA

2. 用有限状态机设计序列检测器

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Bonded IOB (250)	BUFGCTRL (32)
detector	2	4	1	2	2	7	1

Resource	Utilization	Available	Utilization %
LUT	2	20800	0.01
FF	4	41600	0.01
IO	7	250	2.80

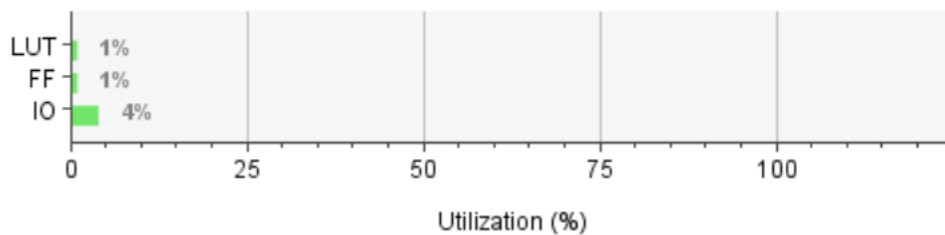


Setup	Hold	Pulse Width
Worst Negative Slack (WNS): inf	Worst Hold Slack (WHS): inf	Worst Pulse Width Slack (WPWS): NA
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): NA
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: NA
Total Number of Endpoints: 12	Total Number of Endpoints: 12	Total Number of Endpoints: NA

3. 用移位寄存器和组合逻辑实现序列检测器

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Bonded IOB (250)	BUFGCTRL (32)
detector2	2	12	4	2	1	10	1

Resource	Utilization	Available	Utilization %
LUT	2	20800	0.01
FF	12	41600	0.03
IO	10	250	4.00



Setup	Hold	Pulse Width
Worst Negative Slack (WNS): inf	Worst Hold Slack (WHS): inf	Worst Pulse Width Slack (WPWS): NA
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): NA
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: NA
Total Number of Endpoints: 42	Total Number of Endpoints: 42	Total Number of Endpoints: NA

五、硬件调试情况

1. 具有异步复位控制的 4bits 十进制同步加法计数器

作为第一个实验，我在熟悉 Vivado 编写流程以及理解约束文件部分花费了较多时间，最初硬件输出中只有 1 位数字显示，原因是我并未把四位八段数码管的 4 个 SEL 管脚写在约束文件中。经过对演示实验的参考和学习，后续调试较为顺利。通过第一个实验，我大致掌握了数逻实验的流程。

2. 用有限状态机设计序列检测器

此实验硬件调试较顺利，在调试过程中出现的问题是指导书中提供的“两个 101011 序

列可以重叠”的样例输出不正确。经过检查发现是状态转移部分的代码有误，在序列匹配后若再输入 0，将转移到编码为 010 的状态（即匹配了子序列“10”的情况）。由此，我也意识到顺利编写和调试程序的关键是对状态转移关系有准确的理解，理论课的内容也是实验课的重要基础。

3. 用移位寄存器和组合逻辑实现序列检测器

最初 LED 可以正确显示移位寄存器的状态，但在匹配成功后检测状态输出时却存在 1 个时钟周期的延时。经过检查发现，由于对移位寄存器和检测状态的判断都进行了非阻塞赋值，检测状态的判断会与移位寄存器的更新同步进行，导致判断的状态实际上是移位寄存器的前一个状态而非当前状态，从而导致了 1 个时钟周期的延时。改用阻塞赋值更新移位寄存器后，问题得到了解决。这也让我对阻塞赋值与非阻塞赋值的区别有了更深入的理解。此外，这也让我更深刻地体会到使用硬件输出可以直观地发现编程中存在的问题。

六、 实验小结

1. 语法注意事项

- (1) case 语句后面不需要冒号
- (2) 端口里的每一句用逗号连接，最后一项不需要任何标点符号；一般语句需要在末尾加分号

2. 约束文件

- (1) 在模块定义中的 input、output 端口均需要引入管脚，其余 reg 寄存器无需引入管脚
- (2) 在引入时钟信号时如果出现报错，可在约束文件中加入一行代码：`set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets {clk_IBUF}]`